

CPSC 488 Assignment 2: Containerized HIGGS MLP on Kubernetes

Investigator	Khoa Do
Date	9/25/2026
Assignment	2: HIGGS MLP on Kubernetes
Class	CPS 488, NLP

Introduction

The assignment attempts to demonstrate the power of utilizing neural networks on a real-world example. The example is taken from the world of particle physics. One of the major efforts in particle physics has been to identify they proposed Higgs boson particle. Physicists have used large devices such as linear particle accelerators, that collide subatomic particles together. These collisions produce events generating many points of data collected by the accelerators for later analysis. The goal in the analysis is to identify the Higgs boson in these collisions from the events that only produce background noise.

Data Format

This assignment uses a HIGGS dataset generated by running a Monte Carlos simulation to simulate the events coming out of a particle accelerator. The row data in this dataset is composed of the following:

The first column is the class label which represents whether this event is a Higgs boson signal or background event.

The next 21 columns represent raw signal values

The last 7 columns represent derived values from the raw signal values.

The dataset rows are organized in the following:

The first 10 million rows are training data used to build the MLP.

The last million rows are for testing the MLP and calculating metrics including training accuracy.

Architecture

The classifier is implemented as a **Multilayer Perceptron (MLP)** with the following structure:

28 input features → 300 neurons → 300 neurons → 100 neurons → 1 output

There's one input layer followed by three hidden layers at 300, 300, 100 neurons each. **ReLU activation function** is used in the hidden layers. This architecture is typical of a funnel architecture where you start with many neurons per hidden layer and constrain the counts as you approach the output layer. The last layer is the one neuron output layer. The value produced is the logit which in the final layer utilizes a **sigmoid activation function** to generate a probability.

During training, the predicted output is compared with the known class label using **binary cross-entropy with logits**. The Adam optimizer then adjusts the network's weights to reduce the classification error. This process is repeated across the training dataset for five epochs.

Training Results

GPU

Device: cuda

GPU: NVIDIA A10

Loading data from: /data/HIGGS.csv.gz

Data loading time: 88.49 seconds

Training examples: 10,000,000

Testing examples: 1,000,000

Required MLP: 28 -> 300 -> 300 -> 100 -> 1

Trainable parameters: 129201

Epoch 1/5 - training loss: 0.561561

Epoch 2/5 - training loss: 0.511975

Epoch 3/5 - training loss: 0.494164

Epoch 4/5 - training loss: 0.485471

Epoch 5/5 - training loss: 0.480144

=== Benchmark Results ===

Data loading seconds: 88.49

Model initialization seconds: 1.35

Training seconds: 674.56

Evaluation seconds: 10.36

Total pipeline seconds: 774.77

Test accuracy: 76.52%

CPU

Device: cpu

Loading data from: /data/HIGGS.csv.gz

Data loading time: 86.67 seconds

Training examples: 10,000,000

Testing examples: 1,000,000

Required MLP: 28 -> 300 -> 300 -> 100 -> 1

Trainable parameters: 129201

Epoch 1/5 - training loss: 0.561544

Epoch 2/5 - training loss: 0.511706

Epoch 3/5 - training loss: 0.493455

Epoch 4/5 - training loss: 0.484934

Epoch 5/5 - training loss: 0.479610

=== Benchmark Results ===

Data loading seconds: 86.67

Model initialization seconds: 4.93

Training seconds: 886.06

Evaluation seconds: 9.55

Total pipeline seconds: 987.20

Test accuracy: 76.55%

Test Accuracy

GPU

Test accuracy: 76.52%

CPU

Test accuracy: 76.55%

The accuracy of both methods were pretty close to each other with any differences being negligible.

Deployment Pipeline

The ML application and libraries was packaged in a Docker image and pushed into the Docker Hub. The image had the ML code, Python, PyTorch. The image was built using the following:

```
docker buildx build --platform linux/amd64 --load \
-t jackdo/cpsc488-higgs:1.0 .
```

We then used a Kubernetes YAML to create a job on the Nautilus cluster using the following:

```
Kubectrl apply -f deployment.yaml
```

We then used kubectrl logs to get the log files.

One challenge encountered during deployment involved persistent storage. A Kubernetes PersistentVolumeClaim remained in the Pending state, preventing the expected storage resource from becoming available. Rather than treating this as an application or MLP failure, the workflow was adjusted so that the required data could be made available to the workload through the working environment.

Benchmark CPU vs GPU Execution

Metric	CPU	GPU
Data loading	86.67 s	88.49 s
Model initialization	4.93 s	1.35 s
Training	886.06 s	674.56 s
Evaluation	9.55 s	10.36 s
Total pipeline	987.20 s	774.77 s

GPU was faster with the major speedup being in the training.

The overall speedup is:

$$Speedup = \frac{T_{cpu}}{T_{gpu}} = \frac{987.20}{774.77} \approx 1.27x$$

Since training was where the main benefit was:

$$Speedup_{Training} = \frac{T_{cpu}}{T_{gpu}} = \frac{886.06}{674.56} \approx 1.31x$$

The main bottleneck was the **model training** which was the significant portion of the execution time. Data loading took time also, but it was roughly the same on GPU and CPU. On a larger dataset, GPU speedup may be significant. With this dataset size, the differences were not as great. Performance was dependent on data transfer from CPU to GPU. That can partially explain some bottleneck. In addition, the batch size was probably enough to take full advantage of the GPU.

Answer the following questions:

- How many trainable parameters does the neural network contain, including biases?
- How many weights does the network contain, excluding biases?
- How many neurons are in the network, excluding the input features?

Parameters = (inputs * output) + biases

$$= (28 * 300) + 300 = 8,700$$

$$(300 * 300) + 300 = 90,300$$

$$(300 * 100) + 100 = 30,100$$

$$(100 * 1) + 1 = 101$$

$$8,700 + 90,300 + 30,100 + 101 = 129,201$$

Number of weights, excluding biases:

$$129,201 - 300 - 300 - 100 - 1 = 128,500$$

Number of neurons, excluding input:

$$300 + 300 + 100 + 1 = 701 \text{ neurons}$$